EPFL | COM-480 DATA VISUALIZATION | SPRING 2026

SPACE ODDITIES

Visualising orbital congestion in the age of mega-constellations.

PROCESS BOOK | **MILESTONE 3**

From the first Sputnik blip in 1957 to 69,055 tracked objects today
a data-driven journey into the orbital traffic jam we built above our heads.

THE CREW

Vincent Fiszbin	SCIPER 394790
Philip P. J. Hamelink	SCIPER 311769
Julien Schluchter	SCIPER 342745

- ▸ **LIVE WEBSITE** com-480-data-visualization.github.io/space-oddities
- ▸ **SOURCE CODE** github.com/com-480-data-visualization/space-oddities
- ▸ **SCREENCAST** www.youtube.com/space-oddities
- ▸ **SUBMITTED** 29 May 2026

KESSLER WAS RIGHT • THE ORBIT OF EARTH IS A LIMITED RESOURCE • MAKE IT VISIBLE

1 / Introduction & Motivation

i INSIGHT | The orbit is full — but no one can see it

In 1978, Donald J. Kessler warned that low Earth orbit could one day become so cluttered that a single collision would trigger a self-sustaining cascade of debris. Today, the global radar catalogue lists **69,055** tracked objects — every piece of hardware registered since Sputnik, including those that have since re-entered the atmosphere — with **32,958** still circling overhead right now. Of those, **12,291** are inert debris, and Starlink alone accounts for over **10,000** active satellites. His prediction is no longer hypothetical. **Space Oddities** translates this invisible traffic jam into something a non-specialist can actually see.

1.1 Problematic

Orbital congestion is a hard story to tell. The relevant data is technical (Two-Line Elements, Conjunction Data Messages), the scales are non-intuitive (hundreds of kilometres at 7 km/s), and the consequences are statistical (a 5.6×10^{-4} collision probability is a number, not a feeling).

At its core, this project is therefore a question of *translation*:

How do we turn rows of orbital mechanics into a story that lands in the gut of someone who has never opened a TLE file?

1.2 Target audience

The general public, science journalists, and policymakers debating space-traffic regulation. We assume *zero* prior knowledge of orbital mechanics and progressively layer in context. Domain experts remain welcome, but they are not who we optimise for.

1.3 The data story (seven chapters).

The site is structured as a guided scrollytelling narrative with seven chapters, each introducing one new angle on the orbital congestion problem.

- **Ch. 1. The Sky Is Getting Crowded.** A dot-per-object orbit canvas with a year slider from 1957 to today, showing the exponential break around 2019.
- **Ch. 2. Where Do They Orbit?** A side-on altitude view of LEO, MEO and GEO with hover-to-highlight orbit bands.
- **Ch. 3. Who Owns Orbit?** Country and operator bar charts expose the privatisation of orbit. SpaceX now operates more satellites than every government combined.
- **Ch. 4. Most of It Is Junk.** A type-breakdown chart and a sunburst toggle show that active payloads are a minority; debris and rocket bodies dominate.
- **Ch. 5. One Collision Can Trigger a Chain Reaction.** An interactive Keplerian orbit simulator lets the user click any satellite and watch a debris cascade propagate.
- **Ch. 6. Near-Misses Happen Every Day.** A table of 2,129 real conjunction events, each linked to an SGP4-computed approach curve and a highway-scale miss animation.
- **Ch. 7. Our Predictions Are Often Wrong.** Covariance error ellipses at closest approach illustrate how positional uncertainty grows with TLE epoch age.

2 / Data Exploration & Preprocessing

2.1 Sources

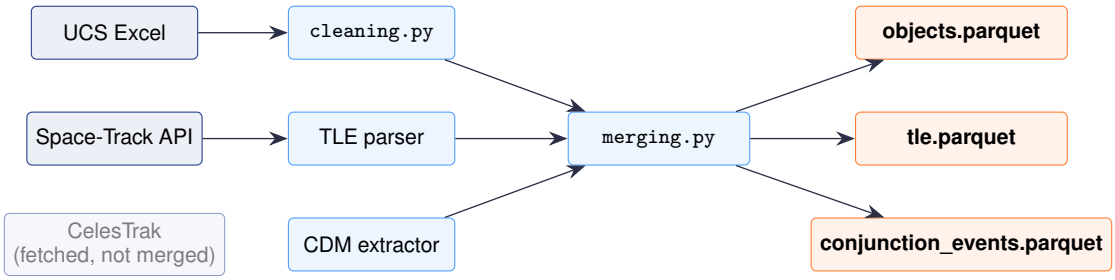
We combine three publicly available registries, each contributing a different facet of the orbital picture.

	UCS Satellite DB	Space-Track.org	CelesTrak
Provides	Rich metadata (purpose, operator, mass, country)	TLEs, SATCAT, Conjunction Data Messages	TLEs, SATCAT (mirror & checks)
Volume	7,560 rows × 27 cols	67,691 TLE entries + CDM feed	69,055 SATCAT objects
Format	Excel	Text + REST API (auth)	CSV / JSON
Freshness	Frozen, May 2023	Updated continuously	Updated daily
Role	Semantic enrichment	Operational ground truth	Cross-validation

Table 1. The three providers underpinning the `objects`, `tle` and `conjunction_events` tables produced by our pipeline.

2.2 Pipeline

A single Python entry-point (`src/main.py`) fetches, cleans, joins and exports the data. The full DAG is summarised below.



2.3 Quality issues we had to solve

- **Stale metadata:** The UCS database was frozen in May 2023; Space-Track is live. Roughly 11% of UCS records pointed to NORAD IDs no longer active. We resolved this with a status join against the live SATCAT, dropping decayed objects and flagging recently launched satellites for which UCS has *no* entry yet.
- **Missing mass / RCS:** About 2% of UCS rows lacked mass; RCS in SATCAT is categorical (SMALL / MEDIUM / LARGE). We chose to keep the categorical bucket rather than impute a continuous mass, since the bucket is what visually matters at our scale.
- **TLE noise:** TLEs include checksum and ephemeris-type fields; we kept only the six classical elements plus epoch and B^* drag term. We propagate positions on-the-fly with the SGP4 model (Python: `sgp4` library; browser-side: `satellite.js`, a JavaScript SGP4 implementation used for the `ApproachChart` and `CovarianceViz`).
- **CDM volume:** Space-Track issues thousands of CDMs per week. We fetch the most recent 30 days (up to 2,000 raw records), then deduplicate to **2,129 events** by retaining one record per object pair (earliest per event). No threshold filter is applied: all events appear in the table; collision probability is shown as "—" when Space-Track did not compute one.

2.4 Headline statistics after cleaning

- **32,958** objects in orbit (24 May 2026 snapshot)
- **17,788** active payloads
- **12,291** debris fragments
- **2,236** spent rocket bodies
- **84.1%** of active payloads in LEO
- **10,371** Starlink satellites in orbit

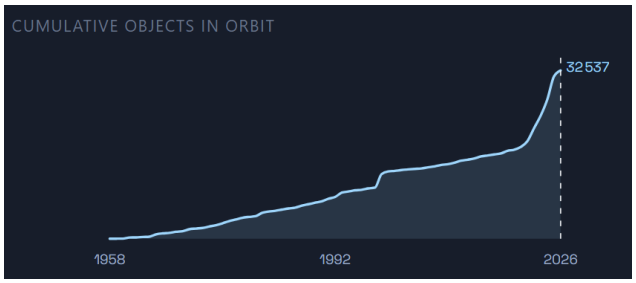


Figure 1. Launches per year, 1957 – 2023. The Starlink era is unmistakable.

All figures above reflect the 24 May 2026 data snapshot. CDMs cover 24 April – 24 May 2026.

3 / Related Work

Real-time trackers. *Stuff in Space* (Yoder) renders every catalogued object as a moving dot in a WebGL globe. Beautiful but mute: a user lands on it and sees a swarm without ever understanding *why* it matters or *what* differentiates a payload from a fragment of a 2007 Chinese ASAT test.

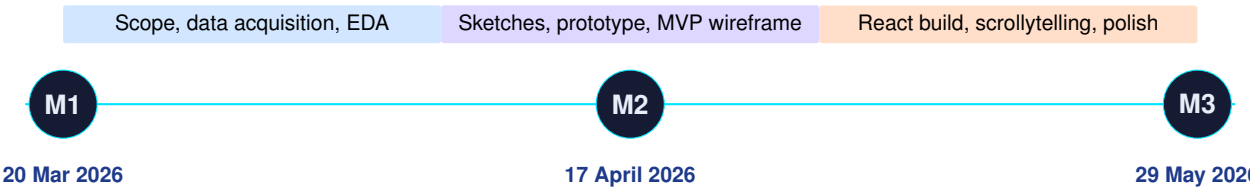
Operational dashboards. *LeoLabs* provides production-grade conjunction monitoring for operators. Excellent for satellite owners, less so for the public: the UI assumes you already understand P_c, time of closest approach and screening volumes.

Editorial pieces. *The Pudding*'s 2017 satellite article and the *Information is Beautiful* satellite chart show what good scrollytelling can do with this domain, but both pre-date the Starlink boom and rely on static snapshots.

ESA density maps. The ESA Space Debris Office publishes one of the most influential visuals in the field: spatial density as a function of altitude. We borrow this idea for our LEO altitude band, but reframe it interactively.

Where Space Oddities sits. We deliberately occupy the gap between the trackers and the dashboards: a *narrative* viewer that uses real, current data but tells a single, opinionated story that orbital space is a finite, shared, and increasingly endangered commons.

4 / Evolution of the Design - from Milestones 1 & 2 to the Final Product



4.1 What we kept, what we cut

The **Milestone 1** scope was ambitious to a fault: real-time 3D globe and manoeuvre planning. Reality forced a triage. We applied a simple rule: *every widget that does not advance the core story is a candidate for the cutting room*. Several ideas were dropped along the way so the final site could stay focused on the orbital congestion narrative.

Originally proposed (M1)	Final outcome (M3)	Why
Full 3D WebGL globe of all 68k objects	2D side-on Earth view + per-orbit cross-section	Performance + clarity; 3D became a swarm, 2D tells the altitude story
Country-level political dashboard	Operator timeline + country small-multiples	Operator (SpaceX, OneWeb…) tells the story better than country in 2026
Stand-alone search/filter page	In-line filters bound to the timeline and orbit views	Removed cognitive overhead; everything in one scroll

Table 2. Scope evolution between M1 and M3. Every cut was made in service of narrative clarity.

4.2 Chapter by chapter: from sketch to scroll

The six M2 wireframes (left column below) survived into the final build, but the chapter list grew by one. Chapters 1, 2, 6 and 7 arrived from their sketches almost untouched. The original “Payloads vs Debris" sketch evolved into a broader composition chapter (Chapter 4 *Most of It Is Junk*, Fig. 8); a brand-new chapter was inserted before it (Chapter 3 *Who Owns Orbit?*, Fig. 6), which has no sketch counterpart, the question emerged naturally while building the project. The original “Kessler Syndrome Events" sketch transformed from a static historical tour into a fully interactive chain-reaction simulator (Chapter 5, Fig. 10). The wireframes shown below are the Milestone 2 iteration, which superseded the Milestone 1 rough sketches; the M2 pass was where interaction patterns and layouts were finalised.

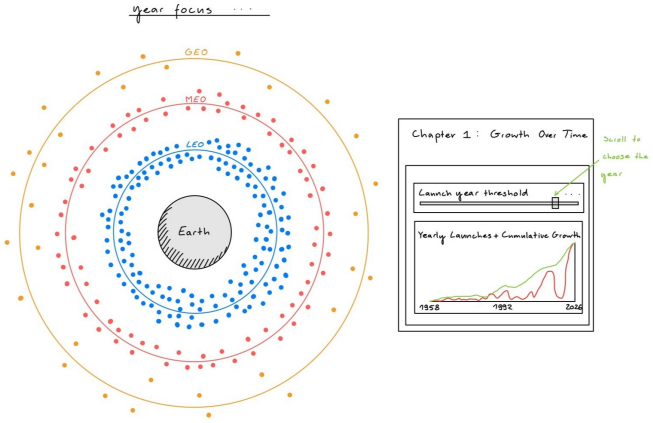


Figure 2. M2 wireframe — *Growth Over Time.* The intended structure: concentric orbital rings, a launch-year slider, and a yearly/cumulative chart beneath.

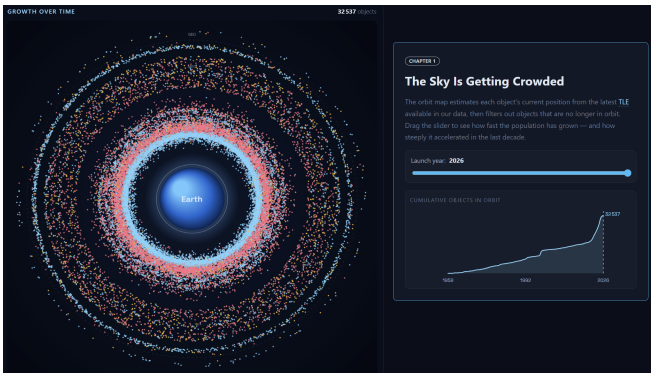


Figure 3. **Chapter 1 — The Sky Is Getting Crowded.** The wireframe’s rings became a live canvas of 32,958 dots; the year slider now drives *both* the orbit population and the cumulative chart below.

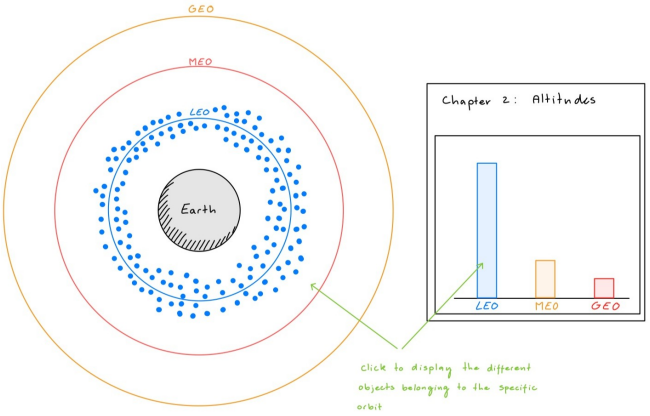


Figure 4. M2 wireframe — *Altitudes.* A bar chart of LEO/MEO/-GEO populations with a click-to-filter promise.

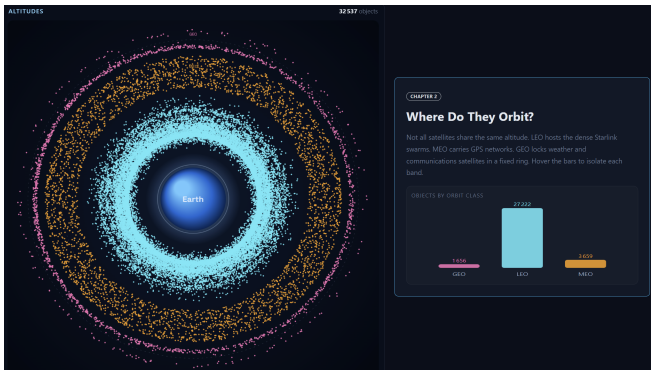


Figure 5. Chapter 2 — Where Do They Orbit? The final version stayed close to the original sketch, replacing the placeholder orbit distributions with real satellite data and implementing a click-to-filter interaction on the bar chart.

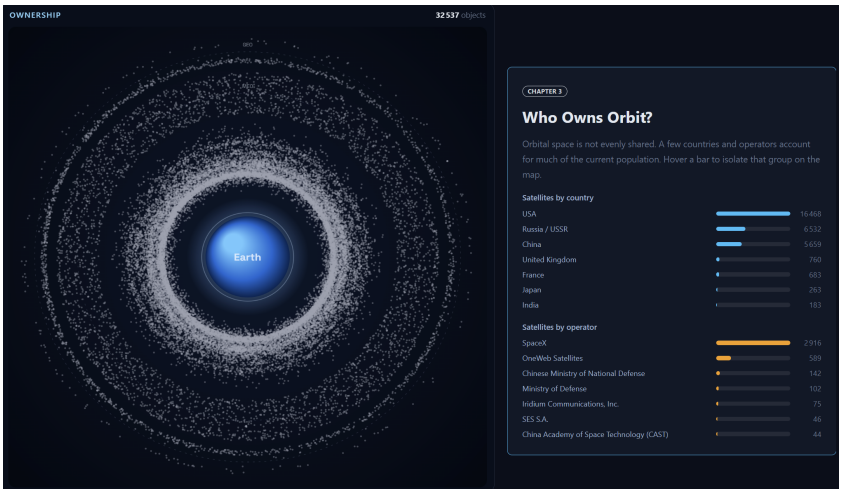


Figure 6. Chapter 3 — Who Owns Orbit? ***(new, no M2 sketch).*** A chapter that emerged mid-build: country and operator small-multiples confront the audience with the most provocative number we found: *SpaceX alone now operates more satellites than every government combined.*

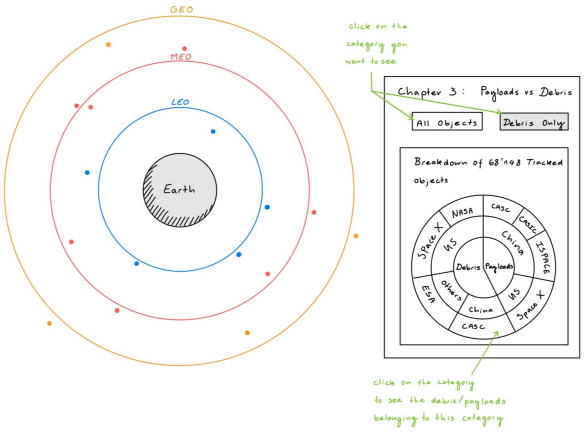


Figure 7. M2 wireframe — *Payloads vs Debris.* A country-segmented composition pie. The country dimension was promoted to its own chapter (Fig. 6); the composition question stayed.

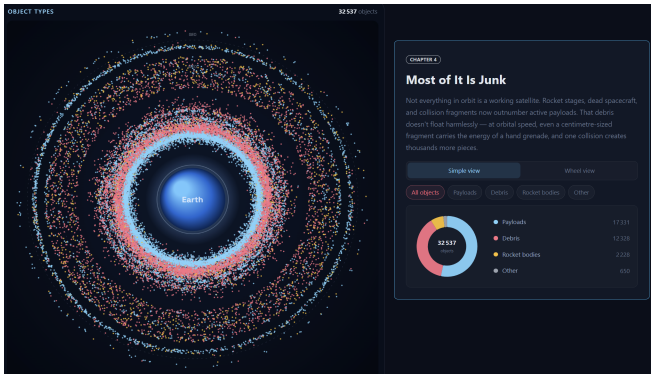


Figure 8. Chapter 4 — Most of It Is Junk. The composition question, sharpened. A type-breakdown bar chart and a sunburst wheel (`TypeOwnershipWheel`) can be toggled to reveal that of $\sim$32,958 tracked objects, active payloads are a minority while debris and rocket bodies dominate.

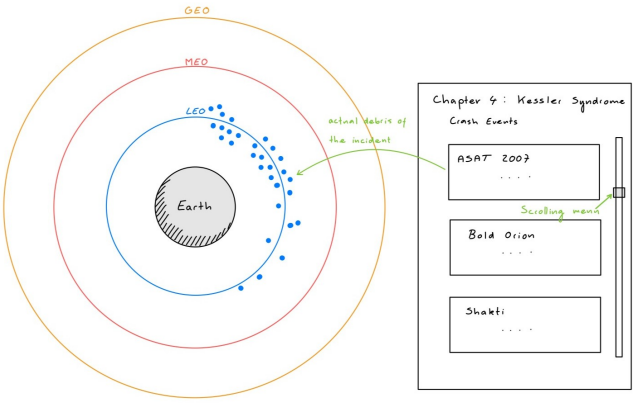


Figure 9. M2 wireframe — *Kessler Syndrome Events.* A static historical tour of documented anti-satellite tests (ASAT 2007, Burnt Frost, Shakti).

Figure 10. Chapter 5 — One Collision Can Trigger a Chain Reaction. We replaced the historical tour with a live simulator: click any satellite to detonate it and watch fragments propagate stochastically. A dual-panel debris-growth chart (`DebrisGrowthChart`) below the simulator contextualises the cascade with real historical data 1957–2024, annotating five key collision events.

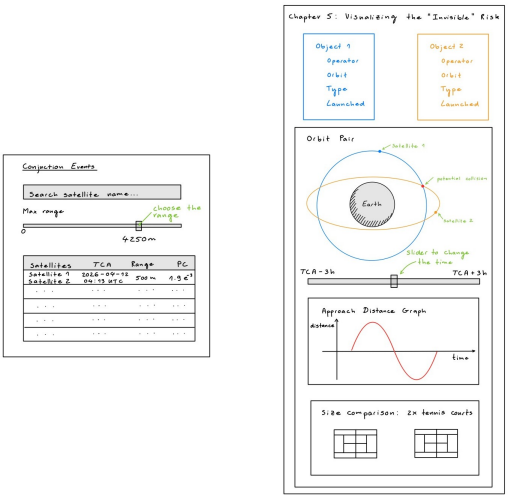


Figure 11. M2 wireframe — *Visualising the Invisible Risk.* A conjunction-events list, an animated orbit-pair view with a Play control, an approach-distance curve, and a sense-of-scale comparison.

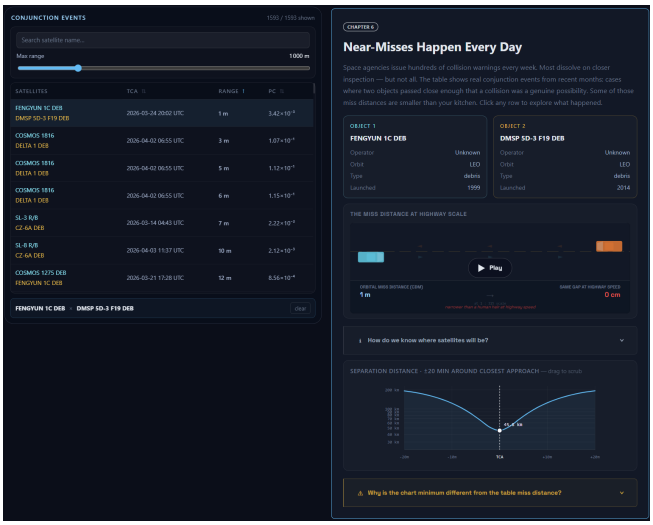
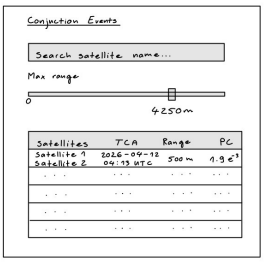


Figure 12. Chapter 6 — Near-Misses Happen Every Day. The most faithful adaptation of the original sketch: real Space-Track CDMs, an animated approach driven by browser-side SGP4, and a distance-vs-time plot.



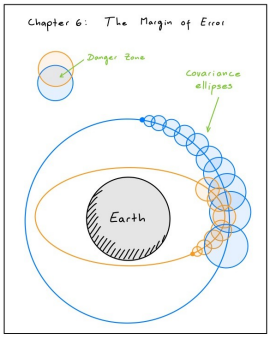


Figure 13. M2 wireframe — *The Margin of Error.* A "Danger Box" and a conjunction ellipsoid sketching the uncertainty around any predicted miss distance.

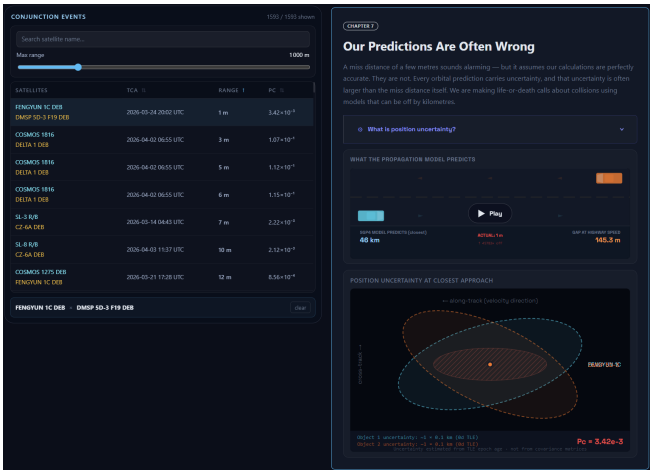
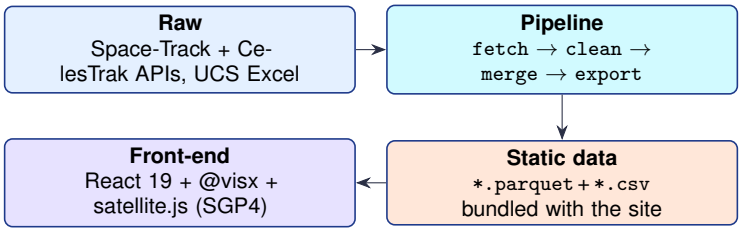


Figure 14. Chapter 7 — Our Predictions Are Often Wrong. The sketched ellipsoid became a real uncertainty cloud showing where each object *might* be at the time of closest approach. The hardest chapter to design and arguably the most important one pedagogically.

5 / Technical Implementation & Architecture

5.1 Stack

The project is split into a **Python data pipeline** (offline, run periodically) and a **React 19 + Vite static front-end** (served via GitHub Pages). Charts are built with **@visx** (a D3-based React charting library) and D3 for imperative canvas drawing. Keeping the front-end static was a deliberate choice: no backend means no operational cost, easy peer-review, and reproducible deploys.



5.2 Front-end architecture

The site is a **single-page React application** (React 19 + Vite) compiled to a static bundle deployed on GitHub Pages. Data is loaded client-side via three React hooks; each chapter is a self-contained component. Key components:

- `OrbitScene.jsx` — Canvas-based orbital map of all $\sim 32{,}958$ in-orbit objects as coloured dots. For each object, the browser uses the latest TLE available in our snapshot and recomputes its current position every few seconds with `satellite.js`, giving a view of orbital motion.
- `KesslerSimulator.jsx` — `canvas`-based Keplerian orbit simulator; click a satellite to trigger a stochastic debris cascade.
- `CdmTable.jsx` — sortable, filterable table of 2,129 conjunction events; clicking a row populates the right-side story panels.
- `ApproachChart.jsx` + `CarAnimationMiss.jsx` — SGP4-computed separation-distance chart and highway-scale miss animation, with the animation and distance plot kept in sync.
- `CovarianceViz.jsx` — SVG error ellipses at closest approach, sized from TLE epoch age.
- `DebrisGrowthChart.jsx` — dual-panel @visx stacked-area chart of debris growth 1957 – 2024 with five annotated collision events.

5.3 Interactivity & UX

The site is structured as a **scrollytelling experience**: the user scrolls, the visualisation reacts. We chose this over a dashboard because the data has a strong narrative arc (historical → current → near-future risk) that benefits from a guided sequence. Inside each act, however, the user is free to explore: hover, brush, filter.

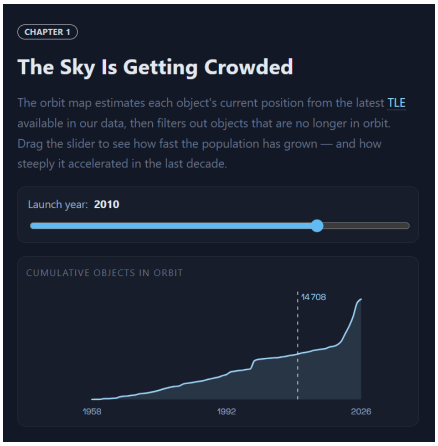


Figure 15. Chapter 1 — *The Sky Is Getting Crowded.* The year slider drives the orbit population and the cumulative chart below

Figure 16. Chapter 4 — *Most of It Is Junk*, simple view. A donut breaks the $\sim$32,537 tracked objects into payloads, debris, rocket bodies and other.

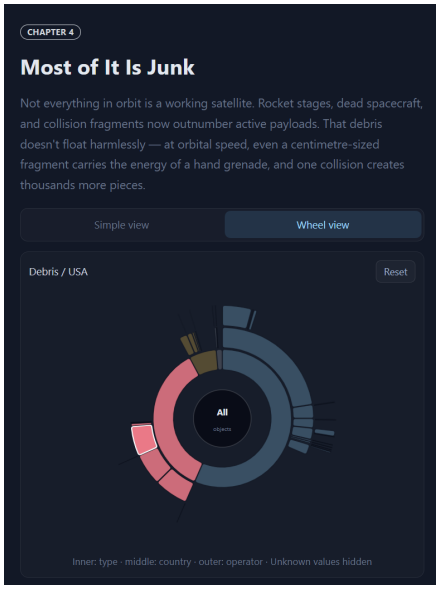


Figure 17. Chapter 4 — *Most of It Is Junk*, wheel view (`TypeOwnershipWheel`). The same population, re-encoded as a sunburst: inner ring = type, middle ring = country, outer ring = operator.

6 / Challenges & Design Decisions

! TECHNICAL CHALLENGE | Rendering thousands of objects without melting the browser

Our first orbit map used the same approach as the early charts: D3 data joins, one SVG circle per object, and normal DOM events. That was fine for a small sample, but it did not scale to thousands of tracked objects refreshing on screen. We moved the dense orbital layer to an HTML5 `canvas`. Each object is propagated from its TLE with `satellite.js`, projected into the compressed orbit view, and drawn as a tiny canvas arc. Propagation runs periodically rather than on every frame; the canvas redraws continuously from the latest positions. D3 still does the surrounding data work: CSV loading, grouping, scales, SVG charts, and quadtree hover detection. Canvas handles the high-volume satellite field; SVG stays where labels, axes, and precise chart interactions matter.

*** DESIGN DECISION | A schematic orbit map, not a literal scale model**

The orbital map is deliberately not drawn at physical scale. Real distances would compress LEO into a thin unreadable ring while pushing GEO to the edge of the canvas. Instead, we map each object into compressed visual bands for LEO, MEO and GEO: the propagated TLE position gives the current angle and orbit class, but the screen radius is adjusted for readability. This was a deliberate visualization choice: we prioritised readability over physical scale.

*** DESIGN DECISION | A scrollytelling spine, not a dashboard**

Early sketches placed every widget on a single screen with a filter sidebar. As the project evolved, this quickly became visually overwhelming. We rebuilt the page as a vertical scroll with chapters, each introducing one new idea at a time. A dashboard encourages free exploration; our goal was instead to guide the reader through a specific narrative about orbital congestion.

! TECHNICAL CHALLENGE | Propagating orbits in the browser, fast enough to animate

For the CDM scene we needed real, physically accurate trajectories, not artistic curves. We ship a small SGP4 implementation client-side. Pre-computing the entire trajectory at page load is very memory-heavy; computing per-frame is too slow. Our compromise: pre-compute 600 sample points per object around the time of closest approach, then linearly interpolate at 60 fps. The error stays below the marker size.

*** DESIGN DECISION | Make a probability *feel* like something**

A CDM with $P_c = 5.6 \times 10^{-4}$ and a 385 m miss distance is meaningless to most of our audience. We sidestep the number entirely: on the page, we render the two objects approaching, plot the predicted miss distance against a scale bar that morphs from *'distance from EPFL BC to BM'* to *'length of an A380'* as the camera zooms. The probability is shown, but only after the geometry has done the emotional work.

! TECHNICAL CHALLENGE | Joining stale UCS to live SATCAT

UCS rows are keyed by NORAD ID; so is SATCAT. Both look clean. They are not: a few thousand UCS records carry leading zeros, padded strings, or international designators in the wrong column. A naive join silently dropped $\sim$12% of payloads. We added a defensive joiner that canonicalises the NORAD ID, then verifies the join against three orthogonal fields (launch date, country, name token), logging any disagreement. The pipeline now keeps a join-quality report alongside the data export.

i INSIGHT | What we learned about telling a hard story

The hardest part of this project was *not* the technical stack. It was deciding what *not* to show. Every cut—the real-time 3D globe, the standalone analytics dashboard, the manoeuvre-planning tool—strengthened the final piece. This project reminded us that visualization is as much about selecting and framing information as it is about rendering it.

7 / Peer Assessment & Team Breakdown

The team divided the work along its natural skill seams: data engineering, visualisation, and editorial / front-end polish. Pair-review was systematic: every pull request required one teammate's approval, and every weekly stand-up reassigned slack so no member became a single point of failure on any deliverable.

Member	Primary contributions	Secondary contributions	Share
Vincent Fiszbin	Data pipeline (`fetch`, `clean`, `merge`); SGP4 / orbit engine; CDM extraction & filtering; deployment workflow	Orbit view canvas rendering; technical sections of the process book	33%
Philip P. J. Hamelink	React components: `KesslerSimulator` (physics engine), `CarAnimationMiss`, `CovarianceViz`, `Hero`, `Team`, `ErrorBoundary`; final data pipeline refresh & deployment	Visual identity & UI/UX styling; scrollytelling layout; M2 prototype	33%
Julien Schluchter	Scrollytelling spine, CDM near-miss panel & physics module; copy-writing and narrative editing; M1 and M3 reports; screencast	D3 interactions on timeline; cross-browser QA; references and bibliography;	33%

Table 3. Breakdown of contributions, agreed by all three team members.

Closing note

Over the course of the project, one thing became increasingly clear to us: orbital congestion is difficult to grasp from raw numbers alone. Tens of thousands of objects, conjunction probabilities, orbital altitudes and TLEs are technically meaningful but not intuitive. The goal of *Space Oddities* was not just to present the data, but to make the structure of the orbital environment visible and understandable to a non-specialist audience. We hope the visitor leaves the site convinced of three things: the orbit is a finite resource; it is being filled faster than it can clear; and the people deciding what happens next would benefit from being able to see what they are deciding about.

Key references. Kessler, D.J. & Cour-Palais, B.G. (1978). *Collision Frequency of Artificial Satellites*, JGR Space Physics 83(A6). Union of Concerned Scientists (2023). *UCS Satellite Database*. US Space Command. *Space-Track.org*. Kelso, T.S. *CelesTrak SATCAT*. Yoder, J. *Stuff in Space*. LeoLabs. *Space Domain Awareness Platform*. The Pudding (2017). *Satellites of the World*. ESA Space Debris Office, *Spatial density of objects by orbital altitude* (2019).